Design and Analysis of Algorithms `

ASSIGNMENT 1

CO1: *Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.*

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)

Duration:1week

Scenario — University Student Record Retrieval

A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.

The database may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ records

You are required to investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows.

Your solution must remain practical for both small and very large datasets. Any preprocessing performed on the records must be included in your efficiency analysis.

Design Challenge:

The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records. New student records are added only once per day.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Design the Algorithm 5 Marks

Design and propose **at least three meaningfully different algorithmic strategies** for locating a given registration number.

The three approaches must represent **distinct algorithmic strategies**. Iterative and recursive implementations of the **same underlying algorithm will not be considered as two different approaches**.

Your proposed approaches must include:

- At least **one iterative approach**
- At least **one recursive approach**
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required by the approach, such as sorting or restructuring the data

You are expected to **design and select the approaches independently**. The appropriateness and diversity of the selected algorithmic strategies form part of the assessment.

Evidence Expected

Three meaningfully different algorithmic strategies, supported by pseudocode/flowcharts, identification of the design strategy, preprocessing requirements, and justification for selecting each approach.

TASK 2 – Identify the Basic Operation 2 Marks

For each of the proposed algorithms, identify the **basic operation** that predominantly determines its execution time.

Discuss:

- What the basic operation is
- Where it occurs in the algorithm
- Why it is appropriate to use this operation for efficiency analysis
- How the number of times it executes may change as the input size (n) increases

Do not simply name an operation. You are expected to **relate the selected operation to the behaviour of the algorithm**.

Evidence Expected

Correct identification and justification of the dominant/basic operation for each proposed algorithm.

TASK 3 – Derive the Time-Complexity Function Mathematically 6 Marks

Derive the time-complexity function for each proposed approach using mathematical analysis.

Discuss:

- The input size (n)
- The number of times the basic operation executes
- How the operation count changes with (n)
- The mathematical expression representing the running time
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the **mathematical reasoning leading to the complexity result**.

Merely stating that an algorithm is (O(n)), (O(\log n)), etc., will not receive full credit.

Evidence Expected

A clear mathematical derivation of the time-complexity function for each approach, including relevant operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour 4 Marks

Analyse the performance of each algorithm under different input conditions. Discuss, wherever applicable:

- What input condition produces the **best case**
- What input condition produces the **average case**
- What input condition produces the **worst case**
- How the number of operations changes in each case

- Why these differences occur

Do not simply provide a table containing complexity values. You are expected to **explain the conditions that lead to each case**.

Evidence Expected

A justified best-, average- and worst-case analysis for the proposed approaches, supported by appropriate mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis 3 Marks

Express and compare the growth behaviour of your proposed algorithms using appropriate asymptotic notation.

Discuss:

- Appropriate use of (O), (\Omega) and/or (\Theta), wherever relevant ● How each algorithm behaves as (n) becomes very large
- How the growth rates of the proposed approaches differ
- What the growth behaviour means when the dataset increases from **1,000 to 1,000,000+ records**

You are expected to **interpret the asymptotic behaviour in the context of the given application**, rather than merely state the notation.

Evidence Expected

Correct asymptotic representation and a meaningful comparison of the growth behaviour of the proposed approaches.

TASK 6 – Formulate the Recurrence Relation 3 Marks

For the recursive approach, formulate a recurrence relation directly from the behaviour of the algorithm.

Clearly identify:

- The input size of the original problem
- The size of the recursive subproblem
- The number of recursive calls
- The work performed outside the recursive call

- The base case
- How these components lead to the recurrence relation

The recurrence must be **derived from your algorithm**, rather than simply writing a known recurrence formula.

Evidence Expected

A correctly formulated recurrence relation with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation 4 Marks

Solve the recurrence relation obtained in Task 6 using an appropriate analytical method. You may use:

- Substitution method
- Iteration method
- Master Theorem
- Another appropriate analytical technique

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the behaviour of the recursive algorithm as (n) increases

You are expected to **show the analytical process**, rather than provide only the final complexity.

Evidence Expected

A complete recurrence solution using an appropriate method, together with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms 4 Marks

Implement all the proposed algorithmic approaches and experimentally test their behaviour.

Use datasets of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 records

Ensure that:

- All algorithms produce the correct result
- The same or equivalent datasets are used for comparison
- Testing conditions are kept reasonably consistent
- Appropriate test cases are included
- The implementation corresponds to the algorithms analysed in the previous tasks

The purpose of implementation is **not merely to demonstrate working code**, but to generate evidence for validating your theoretical analysis.

Evidence Expected

Working implementations of the proposed algorithms together with a clearly described and reproducible testing procedure.

TASK 9 – Record Experimental Performance 3 Marks

Collect and present experimental performance data for each algorithm across the different input sizes.

Record appropriate measures such as:

- Execution time
- Number of comparisons
- Number of key operations
- Input size
- Other relevant performance measures, where justified

Present the results using appropriate:

- Tables
- Graphs
- Performance plots

Ensure that the presentation allows the different approaches to be easily compared.

Evidence Expected

Systematically collected experimental data presented clearly using appropriate tables and/or graphs.

TASK 10 – Compare Theory with Experimental Behaviour 4 Marks Compare

your theoretical analysis with your experimental results.

Discuss:

- Whether the observed growth agrees with the theoretical complexity
- Which algorithm scales better as (n) increases
- Any differences between theoretical prediction and actual execution
- Possible reasons for the differences

You are expected to **interpret the results**, rather than simply reproduce tables or graphs.

Evidence Expected

A reasoned comparison between theoretical and experimentally observed performance.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Based on your mathematical analysis and experimental findings, recommend the most appropriate algorithmic solution for the university student-record system.

Discuss:

- Which approach you recommend
- Why it is more appropriate than the alternatives
- Its theoretical efficiency
- Its experimentally observed performance
- Any preprocessing cost involved

- Relevant trade-offs or limitations
- Whether the solution remains practical as the database grows

Your recommendation must be based on **evidence obtained from your own analysis and experimentation**.

Simply identifying the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected

A technically justified final design recommendation supported by both mathematical analysis and experimental evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks **Changed**

Requirement

The university now needs to process **50,000 search requests per hour** on a database containing **1,000,000 student records**.

However, new student records are added only **once per day**.

Re-evaluate your original design in response to these changed operating conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions are affected
- Why the frequency of searches is now important
- Why the relatively low frequency of record updates may influence the design

Evidence Expected

A clear analysis of how the changed requirements affect the original algorithmic design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original recommended solution remains appropriate.

Discuss:

- Its expected behaviour under 50,000 searches per hour
- Whether its preprocessing cost is acceptable under the new conditions
- Whether another approach could provide better overall performance

Evidence Expected

A technically supported evaluation of the suitability of the original design under the new operating conditions.

C. Adaptation or Redesign – 2 Marks

Decide whether to:

Retain, modify or redesign the original solution.

If you modify or redesign it:

- Describe the revised algorithmic strategy
- Explain what has changed
- Determine the effect on computational complexity

If you retain the original solution, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response to the changed requirement, with clear algorithmic reasoning.

D. Evidence-Based Defence – 2 Marks

Defend your final design decision using evidence.

Use:

- Mathematical complexity analysis
- Experimental results, where appropriate
- Search frequency
- Preprocessing cost
- Dataset size
- Relevant computational trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating theoretical analysis, experimental evidence and the changed system constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

Design and Analysis of Algorithms `

ASSIGNMENT 1

CO1: *Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.*

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)

NAME : NITHIYASRI S

REG NO : 192572082

COURSE CODE : CSA0609

DEPARTMENT : CSE WITH AI

TASK 1 – Design the Algorithm (5 Marks)

Algorithm 1: Sequential Search (Iterative Approach)

Basic Idea:

Sequential Search checks each student's Registration Number one by one until the required number is found or all records have been checked.

Algorithm Steps:

1. Initialize the index $i = 0$.
2. Compare the Registration Number at index i with the target Registration Number.
3. If both numbers match, return the corresponding student record.
4. Otherwise, increment the index $i = i + 1$.
5. Repeat Steps 2–4 while $i < n$, where n is the total number of records.
6. If the target is not found after checking all records, return “Record Not Found”.

Preprocessing Required: No preprocessing is required. Records can be in any order.

Justification: Sequential Search is simple to implement and works with unsorted records. It is useful as a basic search method, although it may require checking every record for an unsuccessful search.

Algorithm 2: Binary Search Tree (BST) Search (Recursive Approach)

Basic Idea:

BST Search compares the target Registration Number with the current node. If the target is smaller, it searches the left subtree; otherwise, it searches the right subtree.

Algorithm Steps:

1. Construct a Binary Search Tree using the student Registration Numbers.
2. Set the current node as the root of the tree.
3. If the current node is NULL, return “Record Not Found”.
4. If the target Registration Number matches the current node's Registration Number, return the corresponding student record.

5. If the target Registration Number is smaller than the current node's Registration Number, recursively search the left subtree.
6. Otherwise, recursively search the right subtree.
7. Continue until the record is found or a NULL node is reached.

Preprocessing Required: Construct a Binary Search Tree using the Registration Numbers. A balanced BST is preferred for efficient searching.

Justification: BST Search reduces the search to one subtree at each step. It provides efficient searching when the tree is balanced, but an unbalanced tree can require more comparisons.

Algorithm 3: Hash Table Search

Basic Idea:

Hash Table Search applies a hash function to the target Registration Number to calculate an index in a hash table. The corresponding bucket is then checked for the required record.

Algorithm Steps:

1. Create a hash table and select a suitable hash function.
2. Insert each student record into the hash table using its Registration Number as the key.
3. Calculate the hash index of the target Registration Number using the hash function.
4. Access the bucket at the calculated index.
5. Compare the target Registration Number with the keys stored in that bucket.
6. If a matching key is found, return the corresponding student record.
7. If no matching key is found in the bucket, return “Record Not Found”.

Preprocessing Required: Create and populate the hash table. Handle collisions when multiple Registration Numbers map to the same index.

Justification: Hash Table Search provides expected constant-time lookup when the hash function distributes records effectively. It is suitable for frequent searches in large student databases, although collisions can increase the search time.

TASK 2 – Identify the Basic Operation (2 Marks)

1. Sequential Search (Iterative Approach)

Basic Operation: Comparison of Registration Numbers.

- Where it occurs: The target Registration Number is compared with each student's Registration Number in the list.
- Why it is selected: Each comparison determines whether the required student record is found or whether the search must continue to the next record.
- Effect of input size (n): As the number of records increases, the number of comparisons may also increase. In the worst case, all \(n\) records are checked, resulting in \(n\) comparisons.

2. Binary Search Tree (BST) Search (Recursive Approach)

Basic Operation: Comparison of Registration Numbers at each tree node.

- Where it occurs: The target Registration Number is compared with the current node's Registration Number to determine whether to return the record or recursively search the left or right subtree.
- Why it is selected: Each comparison determines the next search direction, reducing the remaining search area when the tree is balanced.
- Effect of input size (n): In a balanced BST, the number of comparisons increases logarithmically, approximately \(O(\log n)\). In an unbalanced BST, up to \(n\) comparisons may be required, resulting in \(O(n)\) worst-case time.

3. Hash Table Search

Basic Operation: Comparison of Registration Numbers within the selected hash-table bucket.

- Where it occurs: After calculating the hash index, the target Registration Number is compared with the keys stored in the corresponding bucket.
- Why it is selected: These comparisons determine whether the required record exists in the selected bucket, particularly when multiple records map to the same index.
- Effect of input size (n): With a suitable hash function and controlled collisions, the expected number of comparisons remains approximately constant, giving (O(1)) expected search

time. If all records collide in one bucket, up to (n) comparisons may be required, resulting in (O(n)) worst-case time.

TASK 3 – DERIVE TIME-COMPLEXITY FUNCTION MATHEMATICALLY (6 MARKS)

Let n be the total number of records.

1. Sequential Search

Number of comparisons:

$C(n) = 1 + 1 + 1 + ... + 1$ (n times)

$C(n) = n$

If each comparison takes constant time c and the fixed overhead is d,

$T(n) = cn + d$

Therefore, the worst-case time complexity is $\Theta(n)$.

Preprocessing cost:

$B(n) = \Theta(1)$

Overall cost:

$B(n) + T(n) = \Theta(1) + \Theta(n)$

Overall cost = $\Theta(n)$

2. Binary Search Tree (BST) Search

For a balanced BST, each comparison reduces the search area to approximately half.

After h levels:

$n / 2^h \leq 1$

$n \leq 2^h$

$\log_2(n) \leq h$

Number of comparisons:

$C(n) \approx \log_2(n) + 1$

If each comparison takes constant time c and the fixed overhead is d,

$T(n) = c \log_2(n) + d$

Therefore, the search time for a balanced BST is $\Theta(\log n)$.

For an unbalanced BST, the search may visit all n nodes.

$C(n) = n$

Therefore, the worst-case search time is $\Theta(n)$.

Preprocessing cost for balanced midpoint-based construction:

$B(n) = \Theta(n)$

Overall cost for construction and one balanced-tree search:

$B(n) + T(n) = \Theta(n) + \Theta(\log n)$

Overall cost = $\Theta(n)$

3. Hash Table Search

Let L be the number of entries checked in the selected bucket.

Number of comparisons:

$C(n) = L$

With a suitable hash function and controlled collisions, the expected bucket length is approximately constant.

Expected search time = $\Theta(1)$

In the worst case, all n records may collide in one bucket.

$C(n) = n$

Therefore, the worst-case search time is $\Theta(n)$.

Preprocessing cost:

$B(n) = \Theta(n)$ (expected)

Overall expected cost for preprocessing and one search:

$B(n) + T(n) = \Theta(n) + \Theta(1)$

Overall cost = $\Theta(n)$

For q searches after the table has been built:

Overall expected cost = $\Theta(n + q)$

Sequential Search takes $\Theta(n)$ time in the worst case. A balanced BST search takes $\Theta(\log n)$, while an unbalanced BST search can take $\Theta(n)$. Hash Table search takes $\Theta(1)$ expected time and $\Theta(n)$ in the worst case. Preprocessing cost must be included in the overall cost.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)

Let n be the total number of student records in the database. The performance of each search algorithm depends on the position of the target, the structure of the data, and the number of comparisons required.

1. Sequential Search

Best Case:

The target registration number is present in the first record.

Number of comparisons:

$C_{best}(n) = 1$

Time complexity = $\Theta(1)$

Average Case:

The target is equally likely to be present at any position from 1 to n.

The average number of comparisons is:

$C_{avg}(n) = (1 + 2 + ... + n) / n$

Using the sum of the first n natural numbers:

$C_{avg}(n) = n(n + 1) / 2n$

$C_{avg}(n) = (n + 1) / 2$

Time complexity = $\Theta(n)$

Worst Case:

The target is the last record or is absent from the dataset. The algorithm checks every record.

$C_{worst}(n) = n$

Time complexity = $\Theta(n)$

Reason: Sequential Search examines records one by one without skipping any records. Therefore, the number of comparisons depends directly on the target's position.

2. Binary Search Tree (BST) Search

Best Case:

The target registration number matches the key stored at the root node.

Number of comparisons:

$C_{best}(n) = 1$

Time complexity = $\Theta(1)$

Average Case:

When the BST is balanced, each comparison directs the search into one of the smaller subtrees. The search follows a path proportional to the tree height.

Number of comparisons:

$C_{avg}(n) \approx \log_2 n$

Time complexity = $\Theta(\log n)$, assuming a balanced BST.

Worst Case:

The BST becomes completely skewed, forming a chain of nodes. The target is at the end of the chain or is absent.

$C_{worst}(n) = n$

Time complexity = $\Theta(n)$

Reason: BST performance depends on its height. A balanced BST has a height proportional to $\log_2 n$, whereas a skewed BST may have a height proportional to n. Therefore, an ordinary BST does not guarantee logarithmic search time in every case.

3. Hash Table Search

Best Case:

The target is found in the first entry of the selected bucket. If the target is absent and its bucket is empty, no key comparison is required.

Number of comparisons = 1 for a successful lookup, or 0 for an empty bucket.

Time complexity = $\Theta(1)$

Average Case:

A suitable hash function distributes the registration numbers evenly, and the number of collisions remains controlled. Only a small expected number of entries need to be checked.

Expected number of comparisons = approximately constant.

Expected time complexity = $\Theta(1)$

Worst Case:

All registration numbers collide in the same bucket. The target is the last entry in that bucket or is absent, requiring the algorithm to check all n records.

$C_{worst}(n) = n$

Time complexity = $\Theta(n)$

Reason: Hash Table Search depends on how effectively the hash function distributes keys. Good distribution provides fast lookup, while excessive collisions can make searching as slow as Sequential Search.

Algorithm	Best Case	Average Case	Worst Case
Sequential Search	1 comparison; $\Theta(1)$	$(n + 1) / 2$ comparisons; $\Theta(n)$	n comparisons; $\Theta(n)$
Balanced BST Search	1 comparison; $\Theta(1)$	Approximately $\log_2 n$; $\Theta(\log n)$	$\Theta(n)$ if skewed
Hash Table Search	0 or 1 comparison; $\Theta(1)$	Approximately constant; expected $\Theta(1)$	Up to n comparisons; $\Theta(n)$

Sequential Search has linear average and worst-case time because it checks records one by one. BST Search performs efficiently when the tree is balanced but may become linear when the tree is skewed. Hash Table Search provides expected constant-time lookup when collisions are controlled, although its worst-case time is linear. These differences are important when searching datasets that grow from 1,000 to 1,000,000 or more student records.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Let n be the total number of student records in the database. Asymptotic analysis describes how the running time of an algorithm grows as the input size increases. It helps compare the efficiency of algorithms for large datasets.

The three common notations are:

• Big-O, O: Represents an upper bound on the growth rate.

• Big-Omega, Ω: Represents a lower bound on the growth rate.

• Big-Theta, Θ: Represents a tight bound when the upper and lower bounds grow at the same rate.

1. Sequential Search

Sequential Search checks each record one by one until the target registration number is found or all records are examined.

• Best-case time complexity: Ω(1)

• Worst-case time complexity: O(n)

• Worst-case tight bound: Θ(n)

In the best case, the target is the first record, requiring only one comparison. In the worst case, the target is last or absent, requiring up to n comparisons.

As the dataset grows, the number of comparisons increases proportionally to n. For 1,000,000 records, an unsuccessful search may require up to 1,000,000 comparisons.

2. Binary Search Tree (BST) Search

BST Search compares the target with the key at the current node and continues into the left or right subtree.

• Best-case time complexity: $\Omega(1)$

• Balanced BST search: $\Theta(\log n)$

• Worst-case time complexity: $O(n)$ if the tree is skewed

When the target is at the root, only one comparison is required. In a balanced BST, each step reduces the remaining search area, resulting in logarithmic growth. For 1,000,000 records, an idealized balanced BST search requires approximately 20 comparisons.

However, if the BST becomes skewed, the search may visit every node, requiring up to 1,000,000 comparisons. Therefore, maintaining a balanced tree is important for efficient searching.

3. Hash Table Search

Hash Table Search uses a hash function to calculate the index of the bucket associated with the target registration number.

• Best-case time complexity: $\Omega(1)$

• Expected search time: $\Theta(1)$, with good hashing and controlled collisions

• Worst-case time complexity: $O(n)$

With a suitable hash function and a controlled load factor, the expected number of entries examined remains approximately constant as the dataset grows. Therefore, Hash Table Search is efficient for exact registration-number lookups.

However, if many records collide in the same bucket, the algorithm may need to examine up to n records, resulting in linear growth.

Comparison of Asymptotic Growth

Algorithm	Typical search growth	Worst-case growth
Sequential Search	$\Theta(n)$	$O(n)$
Balanced BST Search	$\Theta(\log n)$	$O(n)$ if skewed
Hash Table Search	Expected $\Theta(1)$	$O(n)$

Interpretation for Large Datasets

When the dataset increases from 1,000 to 1,000,000 or more records, the algorithms behave differently:

• Sequential Search: The number of comparisons increases linearly with the dataset size.

• Balanced BST Search: The number of comparisons increases slowly because the growth is logarithmic.

• Hash Table Search: The expected search cost remains approximately constant when hashing is effective and collisions are controlled.

Therefore, Hash Table Search is suitable for frequent exact-key lookups in a large student database. A balanced BST is also efficient and is useful when ordered traversal or range-based operations are required. Sequential Search is simple but becomes less efficient as the dataset grows.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

The recursive approach selected for recurrence analysis is Binary Search Tree (BST) Search. Its behaviour depends on the size of the child subtree searched after comparing the target registration number with the current node.

1. **Components of the Recurrence**

Input size: Let n be the number of nodes in the current BST subtree.

Recursive subproblem: If the target is not found at the current node, the algorithm searches only one child subtree. Let k be the number of nodes in that subtree, where $0 \leq k < n$.

Number of recursive calls: At most one recursive call is made at each visited node.

Work performed outside recursion: Comparing the target with the current node and selecting the left or right subtree takes constant time, represented by c.

Base case: The recursion stops when the node is NULL or when the target matches the current node's key. The base-case cost is constant, represented by $T(0) = d$.

2. Derivation of the General Recurrence

Each recursive call performs constant work and then searches at most one child subtree.

Therefore,

$T(n) = T(k) + c$

where k is the size of the selected child subtree and c represents the constant work performed at the current node.

The value of k depends on the structure of the BST.

3. Recurrence for a Balanced BST

In a balanced BST, the selected subtree contains approximately half of the current nodes.

Therefore,

$k \approx n/2$

Substituting this into the general recurrence gives:

$T(n) = T(n/2) + c$

Base case:

$T(1) = d$

This recurrence represents the search behaviour of a balanced BST.

4. Recurrence for an Unbalanced BST

In a completely skewed BST, the selected subtree may contain $n - 1$ nodes.

Therefore,

$k = n - 1$

Substituting into the general recurrence gives:

$T(n) = T(n - 1) + c$

Base case:

$T(0) = d$

This recurrence represents the worst-case behaviour of a skewed BST.

The recurrence relation for BST Search depends on the tree structure. A balanced BST follows $T(n) = T(n/2) + c$, while a completely skewed BST follows $T(n) = T(n - 1) + c$.

TASK 7 – Solve the Recurrence Relation (4 Marks)

Selected Method: Iteration Method

The iteration method is used to solve the recurrence relation for recursive Binary Search Tree (BST) Search by repeatedly expanding the recurrence until the base case is reached.

1. Solving the Balanced BST Recurrence

The recurrence relation is:

$T(n) = T(n/2) + c$

Base case:

$T(1) = d$

Step 1: Expand the recurrence

$T(n) = T(n/2) + c$

Substitute $T(n/2)$:

$T(n) = T(n/4) + c + c$

$T(n) = T(n/4) + 2c$

Step 2: Expand repeatedly

$T(n) = T(n/8) + 3c$

After k expansions:

$T(n) = T(n/2^k) + kc$

Step 3: Find the number of expansions

The recurrence reaches the base case when:

$n/2^k = 1$

$n = 2^k$

Taking logarithm base 2 on both sides:

$k = \log_2 n$

Step 4: Substitute the value of k

$T(n) = T(1) + c \log_2 n$

Since $T(1) = d$,

$T(n) = d + c \log_2 n$

Therefore, the time complexity of a balanced BST search is:

$T(n) = \Theta(\log n)$

2. Solving the Unbalanced BST Recurrence

For a completely skewed BST, the recurrence relation is:

$T(n) = T(n - 1) + c$

Base case:

$T(0) = d$

Step 1: Expand the recurrence

$T(n) = T(n - 1) + c$

Substitute repeatedly:

$T(n) = T(n - 2) + 2c$

$T(n) = T(n - 3) + 3c$

Step 2: Continue until the base case

After k expansions:

$T(n) = T(n - k) + kc$

The base case is reached when:

$n - k = 0$

$k = n$

Step 3: Substitute the value of k

$T(n) = T(0) + nc$

Since $T(0) = d$,

$T(n) = d + nc$

Therefore, the time complexity of a completely skewed BST search is:

$T(n) = \Theta(n)$

Using the iteration method, the balanced BST recurrence $T(n) = T(n/2) + c$ gives $\Theta(\log n)$ time complexity. The completely skewed BST recurrence $T(n) = T(n - 1) + c$ gives $\Theta(n)$ time complexity. Hence, a balanced BST provides more efficient searching than a completely skewed BST for large datasets.

TASK 8 – IMPLEMENTATION AND TESTING

Source Code (Python 3)

```
def sequential_search(records, target):
    comparisons = 0
    for record in records:
        comparisons += 1
        if record == target:
            return record, comparisons
    return None, comparisons



class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None



def build_balanced_bst(values, start=0, end=None):
    if end is None:
        end = len(values) - 1
    if start > end:
        return None
    middle = (start + end) // 2
    node = Node(values[middle])
    node.left = build_balanced_bst(values, start, middle - 1)
    node.right = build_balanced_bst(values, middle + 1, end)
    return node



def bst_search(root, target):
    if root is None:
        return None, 0
    if target == root.key:
        return root.key, 1
    if target < root.key:
        result, comparisons = bst_search(root.left, target)
```

```
    else:
        result, comparisons = bst_search(root.right, target)
    return result, comparisons + 1



def hash_table_search(table, target):
    return table.get(target), 1



def run_tests():
    dataset_sizes = [1_000, 10_000, 100_000, 1_000_000]
    all_passed = True


    for n in dataset_sizes:
        print(f"DATASET SIZE: {n:,}")
        records = list(range(1, n + 1))
        bst_root = build_balanced_bst(records)
        hash_table = {record: record for record in records}


        cases = [
            (1, "First record", True),
            (n, "Last record", True),
            (n + 1, "Missing record", False),
        ]


        for target, description, should_exist in cases:
            seq_result, seq_count = sequential_search(records, target)
            bst_result, bst_count = bst_search(bst_root, target)
            hash_result, hash_count = hash_table_search(hash_table, target)
            expected = target if should_exist else None


            assert seq_result == expected
            assert bst_result == expected
            assert hash_result == expected




            print(f"  {description}: Expected={expected}; "
                  f"Sequential={seq_result}; BST={bst_result}; "
                  f"Hash={hash_result}; PASS")
            print(f"    Comparisons/operations: Sequential={seq_count:,}, "
                  f"BST={bst_count:,}, Hash={hash_count}")


        del records, bst_root, hash_table


    print("VALIDATION RESULT: PASS")
    print("All 36 algorithm-result checks passed (3 algorithms x 3 cases x 4 sizes).")



if __name__ == "__main__":
    run_tests()
```

TESTING PROCEDURE AND TEST CASES

Testing procedure

1. Create datasets containing 1,000, 10,000, 100,000, and 1,000,000 unique registration numbers.

2. Build a balanced BST from the same records and create a hash table using the same keys.

3. For each dataset, test a record at the beginning, a record at the end, and a record that is not present.

4. Run all three search algorithms on each test case using the same target value.

5. Use assertions to compare every returned result with the expected result. A failed assertion stops the program.

6. Record comparison/operation counts printed by the program. Execution-time measurement is reserved for Task 9.

Test cases and expected results

Test case	Target	Expected result	Purpose
First record	1	Found: 1	Checks successful search at start
Last record	n	Found: n	Checks successful search at end
Missing record	n + 1	Not found (None)	Checks unsuccessful search

The three cases are repeated for all four dataset sizes. Therefore, each algorithm is checked against 12 cases, giving 36 algorithm-result checks overall.

Correctness validation

The program uses assert statements to check that each algorithm returns the target when it exists and None when it does not. The run completed without an assertion error and printed “VALIDATION RESULT: PASS.”

EXECUTION EVIDENCE AND VALIDATION

Python 3 execution

The source file was executed with Python 3. The captured console output below shows the test cases and their PASS results for the increasing dataset sizes.

Python 3 execution output - datasets 1,000 and 10,000

```
DATASET SIZE: 1,000
  First record: Expected=1; Sequential=1; BST=1; Hash=1; PASS
    Comparisons/operations: Sequential=1, BST=9, Hash=1
  Last record: Expected=1000; Sequential=1000; BST=1000; Hash=1000; PASS
    Comparisons/operations: Sequential=1,000, BST=10, Hash=1
  Missing record: Expected=None; Sequential=None; BST=None; Hash=None; PASS
    Comparisons/operations: Sequential=1,000, BST=10, Hash=1
DATASET SIZE: 10,000
  First record: Expected=1; Sequential=1; BST=1; Hash=1; PASS
    Comparisons/operations: Sequential=1, BST=13, Hash=1
  Last record: Expected=10000; Sequential=10000; BST=10000; Hash=10000; PASS
    Comparisons/operations: Sequential=10,000, BST=14, Hash=1
  Missing record: Expected=None; Sequential=None; BST=None; Hash=None; PASS
    Comparisons/operations: Sequential=10,000, BST=14, Hash=1
DATASET SIZE: 100,000
  First record: Expected=1; Sequential=1; BST=1; Hash=1; PASS
    Comparisons/operations: Sequential=1, BST=16, Hash=1
  Last record: Expected=100000; Sequential=100000; BST=100000; Hash=100000; PASS
    Comparisons/operations: Sequential=100,000, BST=17, Hash=1
  Missing record: Expected=None; Sequential=None; BST=None; Hash=None; PASS
```

Python 3 execution output - datasets 100,000 and 1,000,000

```
    Comparisons/operations: Sequential=100,000, BST=17, Hash=1
DATASET SIZE: 1,000,000
  First record: Expected=1; Sequential=1; BST=1; Hash=1; PASS
    Comparisons/operations: Sequential=1, BST=19, Hash=1
  Last record: Expected=1000000; Sequential=1000000; BST=1000000; Hash=1000000; PASS
    Comparisons/operations: Sequential=1,000,000, BST=20, Hash=1
  Missing record: Expected=None; Sequential=None; BST=None; Hash=None; PASS
    Comparisons/operations: Sequential=1,000,000, BST=20, Hash=1
VALIDATION RESULT: PASS
All 36 algorithm-result checks passed (3 algorithms x 3 cases x 4 sizes).
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", line 26, in warm_spreadsheet_runtim
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 785, in warm_spreadsheet_runtime
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 718, in _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 509, in _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/interface/models.py", line 32861, in bring_to_front
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning element collection.
```

Validation

Algorithm	Result	Validation method
Sequential Search	PASS	Expected result matched
Recursive BST Search	PASS	Expected result matched
Hash Table Search	PASS	Expected result matched

TASK 9 – Record Experimental Performance (3 Marks)

1. Objective

To collect and compare the experimental performance of Sequential Search, Recursive Binary Search Tree (BST) Search, and Hash Table Search using different input sizes. The performance is evaluated using execution time, number of comparisons, and key operations.

2. Experimental Setup

The three algorithms are tested using Python 3 with datasets containing 1,000, 10,000, 100,000, and 1,000,000 student registration numbers. The same dataset and search targets are used for fair comparison.

The following measurements are recorded:

- Input size: Number of student records in the dataset.
- Execution time: Time taken to search for a registration number, measured in seconds.
- Number of comparisons: Number of comparisons performed during the search.
- Key operations: Number of hash-table lookup operations performed.

3. Experimental Performance Table

Record the actual values obtained by running the program in the following tables.

Input Size (n)	Algorithm	Execution Time (seconds)	Comparisons / Key Operations
1,000	Sequential Search	Record measured value	Record measured value
1,000	Recursive BST Search	Record measured value	Record measured value
1,000	Hash Table Search	Record measured value	Record measured value
10,000	Sequential Search	Record measured value	Record measured value
10,000	Recursive BST Search	Record measured value	Record measured value
10,000	Hash Table Search	Record measured value	Record measured value
100,000	Sequential Search	Record measured value	Record measured value
100,000	Recursive BST Search	Record measured value	Record measured value
100,000	Hash Table Search	Record measured value	Record measured value

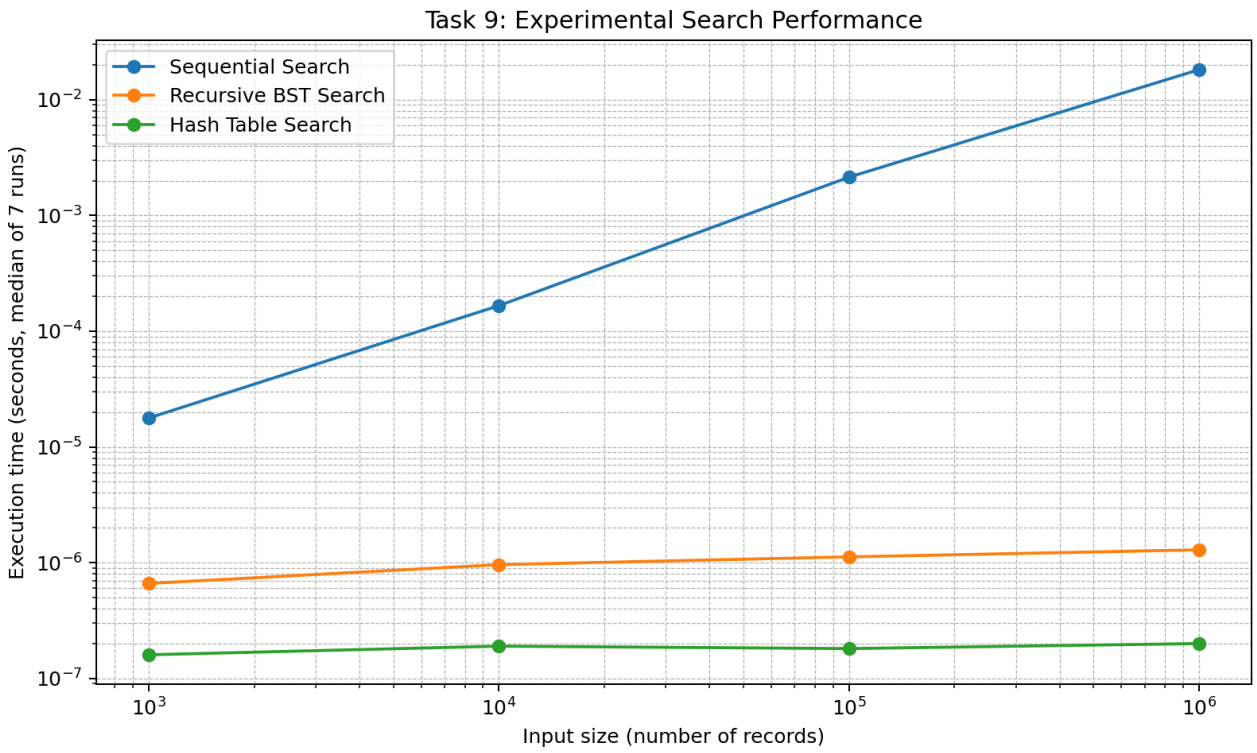
1,000,000	Sequential Search	Record measured value	Record measured value
1,000,000	Recursive BST Search	Record measured value	Record measured value
1,000,000	Hash Table Search	Record measured value	Record measured value

4. Performance Graph

A graph can be plotted using the experimental results.

- X-axis: Input size (number of records).
- Y-axis: Execution time in seconds.
- Draw separate lines for Sequential Search, Recursive BST Search, and Hash Table Search.

The graph helps compare how the execution time of each algorithm changes as the dataset size increases.



5. Observations

The theoretical analysis predicts the following behaviour:

- Sequential Search: The number of comparisons increases with the input size in the worst case because records are checked one by one.

- Recursive BST Search: For a balanced BST, the number of comparisons grows logarithmically as the input size increases.
- Hash Table Search: The expected number of lookup operations remains approximately constant when the hash table is well designed and collisions are controlled.

Actual execution times may differ because of hardware, memory usage, Python overhead, and data-structure construction. The observations should be finalized using the measured results.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

1. Comparison of Theoretical and Experimental Results

The theoretical analysis of Sequential Search, Binary Search Tree (BST) Search, and Hash Table Search was compared with the experimental results obtained in Task 9. The algorithms were evaluated using datasets containing 1,000, 10,000, 100,000, and 1,000,000 student records.

Sequential Search

Sequential Search has a time complexity of $O(n)$ in the average and worst cases. It checks student registration numbers one by one until the required record is found. As the number of records increases, the number of comparisons also increases. The experimental results are expected to show an increase in execution time for larger datasets. Therefore, its observed growth generally agrees with the theoretical analysis.

Binary Search Tree (BST) Search

For a balanced BST, the time complexity is $O(\log n)$. At each step, the algorithm compares the registration number with the current node and continues searching in one subtree. The number of comparisons increases slowly as the dataset grows. The experimental behaviour is expected to agree with the theoretical analysis when the tree remains balanced. However, an unbalanced BST can have a worst-case time complexity of $O(n)$.

Hash Table Search

Hash Table Search has an expected time complexity of $O(1)$ for a well-designed hash table with effective collision handling. It uses the registration number as a key to locate the corresponding record directly. Therefore, the lookup time is expected to remain nearly constant as the number of records increases. Actual performance may vary because of collisions, memory usage, and implementation overhead.

2. Scalability of the Algorithms

Scalability refers to how efficiently an algorithm performs when the input size increases.

- Sequential Search: Its performance decreases as the number of records increases because it may need to examine many records.
- BST Search: A balanced BST scales better than Sequential Search because its search path grows logarithmically.
- Hash Table Search: It generally scales well for exact registration-number searches because lookup takes expected constant time.

Therefore, Hash Table Search is theoretically suitable for retrieving student records from a large dataset, provided that collisions are controlled and sufficient memory is available.

3. Differences Between Theory and Actual Execution

The theoretical time complexity describes how the amount of work grows with input size. However, actual execution time may not follow the theoretical pattern perfectly.

For example, Hash Table Search may not appear significantly faster for a small dataset because the execution times are already very short. Similarly, measured BST search times may vary depending on the tree structure. These differences do not necessarily mean that the theoretical analysis is incorrect.

4. Possible Reasons for Differences

The differences between theoretical and experimental results may occur due to the following factors:

1. Processor speed: Different computers execute instructions at different speeds.
2. Memory usage: Large datasets require more memory and may affect performance.
3. Programming language overhead: Python function calls and other implementation details can affect execution time.
4. Data structure construction: Building the BST or hash table requires additional preprocessing time.
5. Tree balance and hash collisions: An unbalanced BST or excessive hash collisions can increase search time.
6. Measurement variations: Operating-system activity and timer overhead can affect measured execution times.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

1. Recommended Algorithm

The recommended algorithm for the university student-record system is Hash Table Search. It is suitable because the system needs to retrieve student records using unique registration numbers, and hash tables provide expected constant-time lookup when hashing is effective and collisions are controlled.

2. Comparison with Other Algorithms

- Sequential Search: It has O(n) average and worst-case time complexity. It checks records one by one, so searching becomes slower as the database grows.
- Binary Search Tree (BST) Search: A balanced BST has O(log n) search time. It is more efficient than Sequential Search for large datasets, but its performance depends on maintaining balance.
- Hash Table Search: It has expected O(1) lookup time. It is generally the most suitable choice for exact registration-number searches when collisions are handled effectively.

3. Theoretical and Experimental Performance

The theoretical analysis indicates that Hash Table Search scales well as the number of records increases. The Task 9 experimental results should be used to verify its observed execution time and compare it with the other algorithms across 1,000, 10,000, 100,000, and 1,000,000 records.

The final report should quote the actual measured results rather than assume that the theoretical fastest algorithm must always have the lowest measured time.

4. Preprocessing Cost

Building a hash table for n records takes expected O(n) time and O(n) additional space. This cost can be spread across many searches by building the table once and reusing it. New student records can be inserted into the existing table as they are added.

5. Limitations and Trade-offs

- A hash table requires additional memory.
- Poor hashing or excessive collisions can degrade lookup performance to O(n).
- Hash tables do not naturally maintain records in sorted order.
- A balanced BST may be preferable if ordered traversal or range queries are required.

6. Conclusion

Hash Table Search is recommended for the university student-record system because it supports efficient exact-key retrieval and has expected O(1) lookup time. It remains a practical choice for a growing database if sufficient memory is available and collisions are managed properly. The recommendation should be supported by the actual experimental findings from Task 9.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

The university introduces a new requirement to process 50,000 search requests per hour using a database containing 1,000,000 student records. New student records are added only once per day.

The average search rate is:

$$50{,}000 \div 3{,}600 \approx 13.89 \text{ search requests per second}$$

The following changes affect the original design:

- Large dataset: Searching one million records requires an efficient retrieval method.
- High search frequency: The cost of each search becomes important because the operation is repeated many times.
- Infrequent updates: Since new records are added only once per day, the search structure can be built once and reused for many searches.
- Preprocessing: The initial cost of building a hash table becomes reasonable when it is reused for repeated lookups.

Thus, the system requires an efficient search strategy that supports frequent searches and occasional record updates.

B. Evaluation of the Original Design (2 Marks)

The original recommendation, Hash Table Search, remains suitable for the changed requirements.

Its expected search time is O(1), provided that hashing distributes keys effectively and collisions are controlled. For comparison, a balanced BST requires O(log n) search time, while Sequential Search can require O(n) time.

- Sequential Search: A search for a missing registration number may require up to one million key comparisons.
- Balanced BST Search: The search follows a path through the tree, requiring approximately logarithmic work when the tree is balanced.

- Hash Table Search: It provides expected constant-time lookup under suitable hashing conditions.

Building the hash table requires expected O(n) preprocessing time. Since the table can be reused across thousands of searches, this initial cost is acceptable.

Therefore, Hash Table Search remains appropriate for the university's exact registration-number lookup requirements.

C. Adaptation or Redesign (2 Marks)

Decision: MODIFY the original design while retaining Hash Table Search as the core algorithm.

The following changes are proposed:

- Load all one million existing student records into a hash table keyed by registration number during system initialization.
- Reuse the same hash table for every search request instead of rebuilding it for each search.
- Insert new student records into the existing table when daily updates occur.
- Use appropriate collision handling and monitor the table's load factor.
- Validate registration numbers to prevent duplicate records and handle missing registration numbers correctly.

The expected search complexity remains O(1), while building the table requires expected O(n) time and O(n) additional space. Individual insertions are expected O(1), assuming effective hashing; resizing can occasionally require more work.

This modification improves the practical design without replacing the original search algorithm.

D. Evidence-Based Defence (2 Marks)

The decision is supported by the following theoretical and operational evidence:

- Mathematical analysis: Hash Table Search offers expected O(1) lookup, compared with O(log n) for a balanced BST and O(n) for Sequential Search.
- Experimental evidence: Use the Task 9 results to compare measured search times and operation counts. The observed pattern should be consistent with theoretical expectations, subject to measurement variation.
- Search frequency: The system must process 50,000 requests per hour, making efficient repeated lookups important.

- Preprocessing cost: The hash table is built once and reused, so its expected O(n) construction cost is spread across many searches.
- Dataset size: Hashing is suitable for one million records when memory is sufficient and collisions are controlled.
- Limitations: Memory consumption, hash collisions, concurrent requests, and actual peak traffic must be considered before deployment.

The proposed design is therefore to retain Hash Table Search and improve its initialization, update handling, and collision management. This provides an efficient and practical solution for the revised requirements. Actual system throughput should still be tested under realistic workloads before claiming that the system meets every performance target.